

Lab 2 Group 8

Liuxi Mei

Xiaochen Liu

2025-02-03

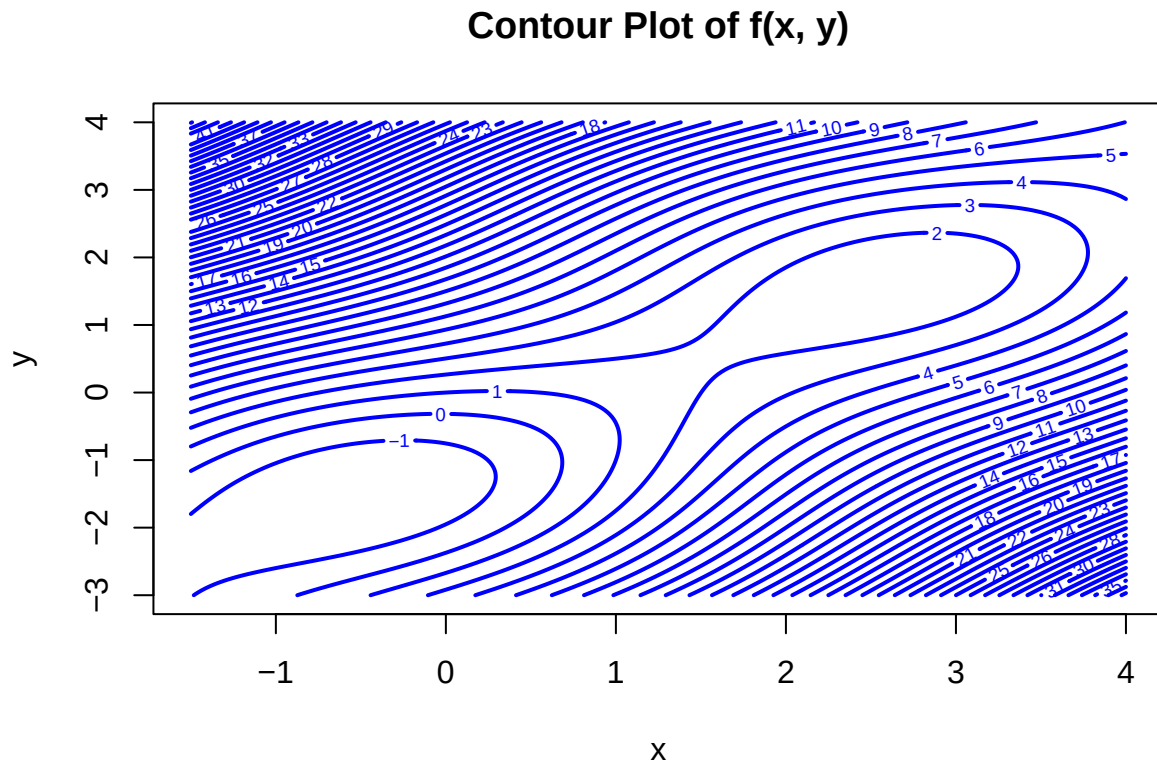
Question 1: Optimization of a two-dimensional function

Consider the two-dimensional function:

$$f(x, y) = \sin(x + y) + (x - y)^2 - 1.5x + 2.5y + 1$$

for $x \in [-1.5, 4]$ and $y \in [-3, 4]$. The aim is to search for the position (x^*, y^*) of the global minimum of f in the given range for x and y .

a) Make a contour plot for this function.



- b) Derive the gradient and Hessian matrix for the function and write R-code for them.
- c) Write your own function applying the Newton algorithm that has the starting value (x_0, y_0) as a parameter
- d) Test several starting values such that you have examples which give at least three different results.

```
## [1] 1.547126
## [1] 0.5471258
## Newton_algorithm gives a minimum value at ( 1.547126 , 0.5471258 at the 6 iteration for a= 2 b= 4
## [1] 2.594471
## [1] 1.594471
## Newton_algorithm gives a minimum value at ( 2.594471 , 1.594471 at the 6 iteration for a= -
1 b= 0
## [1] 1.547197
## [1] 0.5471972
## Newton_algorithm gives a minimum value at ( 1.547197 , 0.5471972 at the 6 iteration for a= -
2 b= -3
## [1] -1.594395
## [1] -2.594395
## Newton_algorithm gives a minimum value at ( -1.594395 , -2.594395 at the 6 iteration for a= 2 b= 0
```

- e) Investigate the candidate results which you have got. Is one of the candidates a global minimum? Which type of solutions are the other candidates?

The last solution gives (-1.59, -2.59) is a global minimum. (1.54, -0.55) is the saddle point. (2.59, 1.59) is a local maximum.

Question 2: Maximum likelihood

Three doses (0.1, 0.3, and 0.9 g) of a drug and placebo (0 g) are tested in a study. Afterward, a dose-dependent event is recorded. The data of $n = 10$ subjects is shown in Table 1; x_i is the dose in grams; $y_i = 1$ if the event occurred, $y_i = 0$ otherwise.

x_i (g)	0	0	0	0.1	0.1	0.3	0.3	0.9	0.9	0.9
y_i	0	0	1	0	1	1	1	0	1	1

Table 1: Data for Question 2

You should fit a simple logistic regression

$$p(x) = P(Y = 1|x) = \frac{1}{1 + \exp(-\beta_0 - \beta_1 x)}$$

to the data, i.e., estimate β_0 and β_1 . One can show that the log likelihood is

$$g(b) = \sum_{i=1}^n [y_i \log \{(1 + \exp(-\beta_0 - \beta_1 x_i))^{-1}\} + (1 - y_i) \log \{1 - (1 + \exp(-\beta_0 - \beta_1 x_i))^{-1}\}]$$

where $b = (\beta_0, \beta_1)^T$ and the gradient is

$$g'(b) = \sum_{i=1}^n \left[y_i - \frac{1}{1 + \exp(-\beta_0 - \beta_1 x_i)} \right] \begin{pmatrix} 1 \\ x_i \end{pmatrix}$$

““

a. Write a function for an ML-estimator for (β_0, β_1) using the steepest ascent method with a step-size-reducing line search (back-tracking). For this, you can use and modify the code for the steepest ascent example of the lecture. The function should count the number of function and gradient evaluations.

```
## [1] -0.009353495  1.262798921
## Steepest ascent method gives result -0.009353495 1.262799 at the 38 th iteration for -
0.2 1
```

b. Compute the ML-estimator with the function from a. for the data (x_i, y_i) above. Use a stopping criterion such that you can trust five digits of both parameter estimates for β_0 and β_1 . Use the starting value $(\beta_0, \beta_1) = (-0.2, 1)$. The exact way to use backtracking can be varied. Try two variants and compare the number of function and gradient evaluations performed to convergence.

```
## [1] -0.009353495  1.262798921
## Steepest ascent method gives result -0.009353495 1.262799 at the 38 th iteration for -
0.2 1
```

```
## [1] -0.009286907  1.262506751
## Steepest ascent method gives result -0.009286907 1.262507 at the 33 th iteration for 2 1
```

```
model_BFGS <- optim(c(-0.2,1), g, gr=gradient, method='BFGS')
print(model_BFGS$par)
```

c. Use now the function `optim` with both the BFGS and the Nelder-Mead algorithm. Do you obtain the same results as for b.? Is there any difference in the precision of the result? Compare the number of function and gradient evaluations that are given in the standard output of `optim`.

```
## [1] -516.8771 -214.3395
```

```
model_nelder <- optim(c(-0.2,1), g, method='Nelder-Mead')
print(model_nelder$par)
```

```
## [1] -698.17286 -12.89984
```

```
model_glm <- glm(y ~ x, family = binomial(link = "logit"))
summary(model_glm)
```

d. Use the function `glm` in R to obtain an ML-solution and compare it with your results before.

```
##
## Call:
## glm(formula = y ~ x, family = binomial(link = "logit"))
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.00936    0.87086  -0.011   0.991
## x           1.26282    1.86663   0.677   0.499
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 13.460  on 9  degrees of freedom
## Residual deviance: 12.969  on 8  degrees of freedom
## AIC: 16.969
##
## Number of Fisher Scoring iterations: 4
```